

Dynamic Memory Allocation

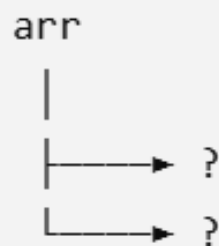
Step 1

⟨/⟩ C++

```
int ***arr = new int**[layer];
```

अगर `layer = 2`

तो memory



अभी सिर्फ 2 pointers बने हैं।

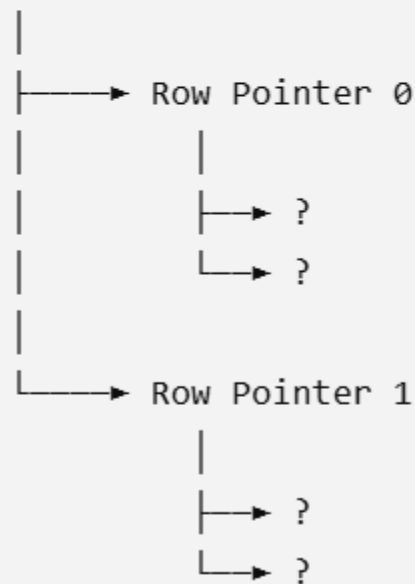
Step 2

</> C++

```
for(int i=0;i<layer;i++)  
    arr[i]=new int*[row];
```

अगर row = 2

arr



अब हर layer के अंदर row pointers बन गए।



Step 3

</> C++

```
for(int j=0;j<row;j++)  
    arr[i][j]=new int[col];
```

अगर col = 3

Layer 0

Row0

```
+---+---+---+  
|   |   |   |  
+---+---+---+
```

Row1

```
+---+---+---+  
|   |   |   |  
+---+---+---+
```

Layer1

Multi dymasional array

Row0

```
+---+---+---+
|   |   |   |
+---+---+---+
```

Row1

```
+---+---+---+
|   |   |   |
+---+---+---+
```

Layer1

Row0

```
+---+---+---+
|   |   |   |
+---+---+---+
```

Row1

```
+---+---+---+
|   |   |   |
+---+---+---+
```

अब पूरा 3D array बन गया।

Input

`</> C++`

```
for(int i=0;i<layer;i++)  
    for(int j=0;j<row;j++)  
        for(int k=0;k<col;k++)  
            cin>>arr[i][j][k];
```

यदि input

```
1 2 3  
4 5 6  
7 8 9  
10 11 12
```

तो memory

disp() Function

</> C++

```
void disp(int ***arr,int layer,int row,int col)
{
    for(int i=0;i<layer;i++)
    {
        cout<<"Layer "<<i<<endl;

        for(int j=0;j<row;j++)
        {
            for(int k=0;k<col;k++)
            {
                cout<<arr[i][j][k]<<" ";
            }
            cout<<endl;
        }

        cout<<endl;
    }
}
```

अगर

Layer = 2

Row = 3

Col = 4

तो array होगा:

⟨⟩ C++

```
int arr[2][3][4];
```

मतलब:

- 2 Layers
- हर Layer में 3 Rows
- हर Row में 4 Columns

कुल elements:

$$2 \times 3 \times 4 = 24$$

Memory Diagram

```
arr
|
|— Layer 0
|   |
|   |— Row 0 → [1] [2] [3] [4]
|   |— Row 1 → [5] [6] [7] [8]
|   |— Row 2 → [9] [10] [11] [12]
|
|— Layer 1
|   |
|   |— Row 0 → [13] [14] [15] [16]
|   |— Row 1 → [17] [18] [19] [20]
|   |— Row 2 → [21] [22] [23] [24]
```


$i = 0$ (Layer 0)

$j = 0$ (Row 0)

$k = 0 \rightarrow \text{arr}[0][0][0]$

$k = 1 \rightarrow \text{arr}[0][0][1]$

$k = 2 \rightarrow \text{arr}[0][0][2]$

$k = 3 \rightarrow \text{arr}[0][0][3]$

$j = 1$ (Row 1)

$\text{arr}[0][1][0] \dots \text{arr}[0][1][3]$

$j = 2$ (Row 2)

$\text{arr}[0][2][0] \dots \text{arr}[0][2][3]$

$i = 1$ (Layer 1)

$j = 0$

$\text{arr}[1][0][0] \dots \text{arr}[1][0][3]$

$j = 1$

$\text{arr}[1][1][0] \dots \text{arr}[1][1][3]$

$j = 2$

$\text{arr}[1][2][0] \dots \text{arr}[1][2][3]$

Multi dymasional array

```
int main() {
    int layer, row, col;

    cout << "Enter layer row col: ";
    cin >> layer >> row >> col;

    // Allocate memory
    int ***arr = new int**[layer];

    for (int i = 0; i < layer; i++) {
        arr[i] = new int*[row];

        for (int j = 0; j < row; j++) {
            arr[i][j] = new int[col];
        }
    }

    input(arr, layer, row, col);
    display(arr, layer, row, col);
}
```

```
// Delete memory
for (int i = 0; i < layer; i++) {
    for (int j = 0; j < row; j++) {
        delete[] arr[i][j];
    }
    delete[] arr[i];
}

delete[] arr;
arr = NULL;

return 0;
```

अगर 5D array के dimensions के meaningful नाम रखने हैं, तो ऐसे रख सकते हैं:

- layer
- row
- col
- depth
- time (या page, block आदि कोई भी नाम)

उदाहरण:

```
arr[layer][row][col][depth][time]
```

```
arr[2][3][4][5][6]
```

i → layer (2)

j → row (3)

k → col (4)

l → depth (5)

m → time (6)

Multi dymasional array

```
void disp(int arr[][3][4][5][6], int layer, int row, int col, int depth, int time)
{
    for(int i = 0; i < layer; i++)
    {
        for(int j = 0; j < row; j++)
        {
            for(int k = 0; k < col; k++)
            {
                for(int l = 0; l < depth; l++)
                {
                    for(int m = 0; m < time; m++)
                    {
                        cout << arr[i][j][k][l][m] << " ";
                    }
                    cout << endl;
                }
                cout << endl;
            }
            cout << endl;
        }
        cout << "-----" << ↓ 1;
    }
}
```

```
int main()
{
    int layer = 2;
    int row    = 3;
    int col    = 4;
    int depth  = 5;
    int time   = 6;

    int arr[2][3][4][5][6];

    int x = 1;

    for(int i = 0; i < layer; i++)
        for(int j = 0; j < row; j++)
            for(int k = 0; k < col; k++)
                for(int l = 0; l < depth; l++)
                    for(int m = 0; m < time; m++)
                        arr[i][j][k][l][m] = x++;

    disp(arr, layer, row, col, depth, time)
```



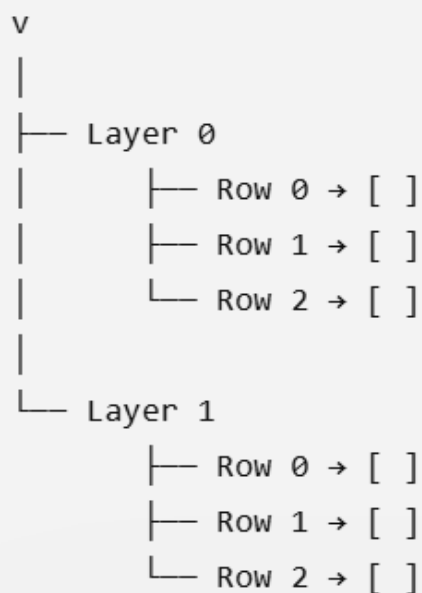
3D Vector का मतलब है vector के अंदर vector, उसके अंदर vector।

Syntax:

`</> C++`

```
vector<vector<vector<int>>> v;
```

इसे ऐसे समझो:




```
int main()
{
    int layer = 2;
    int row = 3;
    int col = 4;

    vector<vector<vector<int>>> v(
        layer,
        vector<vector<int>>(
            row,
            vector<int>(col)
        )
    );

    int x = 1;

    // Input
    for(int i=0;i<layer;i++)
        for(int j=0;j<row;j++)
            for(int k=0;k<col;k++)
                v[i][j][k] = x++;
```

```
int layer, row, col;
cin >> layer >> row >> col;

vector<vector<vector<int>>> v(
    layer,
    vector<vector<int>>(row, vector<int>(col))
);
```

```
void disp(vector<vector<vector<int>>> &v, int layer, int row, int col)
{
    for(int i = 0; i < layer; i++)
    {
        cout << "Layer " << i << endl;

        for(int j = 0; j < row; j++)
        {
            for(int k = 0; k < col; k++)
            {
                cout << v[i][j][k] << " ";
            }
            cout << endl;
        }
        cout << endl;
    }
}
```

```
int main() {  
    int n;  
    cout << "Enter size: ";  
    cin >> n;  
  
    // Dynamic array  
    int *arr = new int[n];  
  
    input(arr, n);  
    display(arr, n);  
  
    delete[] arr;  
    arr = NULL;  
  
    return 0;  
}
```

```
cout << "Enter rows and cols: ";
cin >> row >> col;

// Create 2D dynamic array
int **arr = new int*[row];

for (int i = 0; i < row; i++) {
    arr[i] = new int[col];
}

input(arr, row, col);
display(arr, row, col);

// Delete 2D array
for (int i = 0; i < row; i++) {
    delete[] arr[i];
}
delete[] arr;
arr = NULL;

return 0;
```

```
void input(int **arr, int row, int col) {  
    cout << "Enter elements:\n";  
    for (int i = 0; i < row; i++) {  
        for (int j = 0; j < col; j++) {  
            cin >> arr[i][j];  
        }  
    }  
}  
  
void display(int **arr, int row, int col) {  
    cout << "Matrix:\n";  
    for (int i = 0; i < row; i++) {  
        for (int j = 0; j < col; j++) {  
            cout << arr[i][j] << " ";  
        }  
        cout << endl;  
    }  
}
```

1. Insert at End

</> C++

```
#include <iostream>
using namespace std;

void insertEnd(int arr[], int &n, int cap, int val) {
    if (n == cap) {
        cout << "Array Full\n";
        return;
    }

    arr[n] = val;
    n++;
}

int main() {
    int arr[10] = {10,20,30,40};
    int n = 4;

    insertEnd(arr, n, 10, 50);

    for(int i=0;i<n;i++)
        cout<<arr[i]<<" ";
```



2. Insert at Head

</> C++

```
#include <iostream>
using namespace std;

void insertHead(int arr[], int &n, int cap, int val) {
    if (n == cap) {
        cout << "Array Full\n";
        return;
    }

    for (int i = n-1; i >= 0; i--) {
        arr[i+1] = arr[i];
    }

    arr[0] = val;
    n++;
}

int main() {
    int arr[10] = {20,30,40};
    int n = 3;
```



3. Insert at Index

```
</> C++

#include <iostream>
using namespace std;

void insertAtIdx(int arr[], int &n, int cap, int idx, int val) {

    if (idx < 0 || idx > n) {
        cout << "Invalid Index\n";
        return;
    }

    if (n == cap) {
        cout << "Array Full\n";
        return;
    }

    for (int i = n-1; i >= idx; i--) {
        arr[i+1] = arr[i];
    }

    arr[idx] = val;
    n++;
}
```



1. Delete at Head

```
</> C++

void deleteHead(int arr[], int &n) {
    if (n == 0) {
        cout << "Array Empty\n";
        return;
    }

    int del = arr[0];

    for (int i = 1; i < n; i++) {
        arr[i - 1] = arr[i];
    }

    n--;

    cout << "Deleted = " << del << endl;
}
```

2. Delete at End

`</> C++`

```
void deleteEnd(int arr[], int &n) {  
    if (n == 0) {  
        cout << "Array Empty\n";  
        return;  
    }  
  
    int del = arr[n - 1];  
  
    n--;  
  
    cout << "Deleted = " << del << endl;  
}
```

3. Delete at Index

`</> C++`

```
void deleteAtIdx(int arr[], int &n, int idx) {  
    if (n == 0) {  
        cout << "Array Empty\n";  
        return;  
    }  
  
    if (idx < 0 || idx >= n) {  
        cout << "Invalid Index\n";  
        return;  
    }  
  
    int del = arr[idx];  
  
    for (int i = idx + 1; i < n; i++) {  
        arr[i - 1] = arr[i];  
    }  
  
    n--;  
  
    cout << "Deleted = " << del << endl;  
}
```



1. Insert at Head

`</> C++`

```
void insertHead(vector<int> &v, int val) {  
    v.insert(v.begin(), val);  
}
```

2. Insert at End

`</> C++`

```
void insertEnd(vector<int> &v, int val) {  
    v.push_back(val);  
}
```

3. Insert at Index

`</>` C++

```
void insertAtIdx(vector<int> &v, int idx, int val) {
    if (idx < 0 || idx > v.size()) {
        cout << "Invalid Index\n";
        return;
    }

    v.insert(v.begin() + idx, val);
}
```

Time Complexity

Operation	Function	Time
Insert at Head	<code>v.insert(v.begin(), val)</code>	$O(n)$
Insert at End	<code>v.push_back(val)</code>	$O(1)$ (amortized)
Insert at Index	<code>v.insert(v.begin()+idx, val)</code>	$O(n)$

1. Delete at Head

`</> C++`

```
void deleteHead(vector<int> &v) {  
    if (v.empty()) {  
        cout << "Vector Empty\n";  
        return;  
    }  
  
    int del = v.front();  
    v.erase(v.begin());  
  
    cout << "Deleted = " << del << endl;  
}
```

2. Delete at End

`</> C++`

```
void deleteEnd(vector<int> &v) {  
    if (v.empty()) {  
        cout << "Vector Empty\n";  
        return;  
    }  
  
    int del = v.back();  
    v.pop_back();  
  
    cout << "Deleted = " << del << endl;  
}
```

3. Delete at Index

⟨/⟩ C++

```
void deleteAtIdx(vector<int> &v, int idx) {  
    if (v.empty()) {  
        cout << "Vector Empty\n";  
        return;  
    }  
  
    if (idx < 0 || idx >= v.size()) {  
        cout << "Invalid Index\n";  
        return;  
    }  
  
    int del = v[idx];  
    v.erase(v.begin() + idx);  
  
    cout << "Deleted = " << del << endl;  
}
```


`</> C++`

```
HashTable h;  
  
h.insert(10);  
h.insert(17);  
h.insert(24);  
h.insert(31);  
h.insert(38);  
h.insert(39);
```

अब हर insertion के बाद table देखो।

Index	Value	
-----	-----	
0	38	←
1	39	
2	Empty	
3	10	
4	17	
5	24	Primary Cluster
6	31	←

अब समझो Primary Clustering

शुरुआत में सिर्फ 10 था।

3 → 10

फिर 17 आया, उसी cluster से टकराया।

फिर 24 आया, cluster और बड़ा हुआ।

फिर 31 आया, फिर 38, फिर 39।

यानी एक छोटा cluster:

3 → 10

धीरे-धीरे बड़ा होकर बन गया:

0 → 38
1 → 39
2 → Empty
3 → 10
4 → 17
5 → 24
6 → 31

अब यदि 45 insert करो:

$$45 \% 7 = 3$$

तो उसे check करना पड़ेगा:

3 ✗
4 ✗
5 ✗
6 ✗
0 ✗
1 ✗
2 ✓



यानी 6 collisions हुईं।

यही Primary Clustering की समस्या है—एक cluster बनता है और हर नई key उसी cluster को और बड़ा करती जाती है।

ff-e3e7d81e4186

इसमें याद रखने वाली बात

- `idx` = Hash Function निकालता है।

```
</> C++
```

```
idx = key % m;
```

उदाहरण:

```
10 % 7 = 3
```

- `newidx` = Collision होने पर नई जगह ढूंढता है।

```
</> C++
```

```
newidx = (idx + i) % m;
```

पहली बार (`i = 0`):

```
newidx = (3 + 0) % 7 = 3
```



```
</> C++
```

```
newidx = (idx + i*i) % 7;
```

अब insert करते हैं

```
</> C++
```

```
h.insert(10);
```

```
h.insert(17);
```

```
h.insert(24);
```

```
h.insert(31);
```

Step 1 : `h.insert(10)`

```
10 % 7 = 3
```

Step 2 : `h.insert(17)`

$$17 \% 7 = 3$$

Quadratic Probing

$$i=0$$

$$\begin{aligned} \text{newidx} &= (3 + 0^2) \% 7 \\ &= 3 \quad \times \end{aligned}$$

$$i=1$$

$$\begin{aligned} \text{newidx} &= (3 + 1^2) \% 7 \\ &= 4 \quad \checkmark \end{aligned}$$

Index	Value
-----	-----
0	Empty
1	Empty
2	Empty
3	10
4	17
5	Empty
6	Empty

Step 4 : `h.insert(31)`

$$31 \% 7 = 3$$

अब फिर वही sequence

$i=0 \rightarrow 3$ ❌

$i=1 \rightarrow 4$ ❌

$i=2 \rightarrow 0$ ❌

$i=3 \rightarrow (3+9)\%7=5$ ✅

Index	Value
-----	-----
0	24
1	Empty
2	Empty
3	10
4	17
5	31
6	Empty

$$10 \% 7 = 3$$

$$17 \% 7 = 3$$

$$24 \% 7 = 3$$

$$31 \% 7 = 3$$

इसलिए चारों ने **एक ही** Probe Sequence follow किया।

$$\text{Hash} = 3$$

3 → 4 → 0 → 5 → ...

↑ ↑ ↑ ↑

| | | |

10 17 24 31

यही **Secondary Clustering** है।

Secondary Clustering क्यों?

क्योंकि

10

17

24

31

इनका Initial Hash एक ही (3) है।

इसलिए ये एक ही probing sequence follow कर रहे हैं।

```
10 ┌  
17 ┌  
24 ┌─→ 3 → 4 → 0 → 5 ...  
31 └
```

Double Hashing का मतलब है कि collision होने पर next index खोजने के लिए दूसरा hash function इस्तेमाल किया जाता है।

इसलिए इसे सबसे अच्छा Open Addressing method माना जाता है क्योंकि इसमें Primary Clustering और Secondary Clustering दोनों लगभग खत्म हो जाती हैं।

$$h1(key) = key \% m$$
$$h2(key) = \text{prime} - (key \% \text{prime})$$

जहाँ,

m = Hash Table Size

prime = ऐसा Prime Number जो m से छोटा हो।
अर्थात् $\text{prime} < m$

```
</> C++
```

```
HashTable h;  
  
h.insert(10);  
h.insert(17);  
h.insert(24);  
h.insert(31);
```

क्यों?

क्योंकि

```
10 % 7 = 3  
17 % 7 = 3  
24 % 7 = 3  
31 % 7 = 3
```

सभी का **First Hash** (`h1`) = 3 आएगा।

अब **Second Hash** (`h2`) अलग-अलग होगा।



सभी का **First Hash** (h_1) = 3 आएगा।

अब **Second Hash** (h_2) अलग-अलग होगा।

Key	$h_1 = \text{key} \% 7$	$h_2 = 5 - (\text{key} \% 5)$
10	3	5
17	3	3
24	3	1
31	3	4

इसलिए इनका probing path अलग-अलग होगा।

इसलिए इनका probing path अलग-अलग होगा।

10 → 3

17 → 3 → 6

24 → 3 → 4

31 → 3 → 0

Advantages (Primary Clustering / Linear Probing)	Disadvantages	
सरल (Simple) implementation	Primary Clustering की समस्या होती है।	
Insert, Search, Delete का code आसान होता है।	लगातार (continuous) cluster बन जाते हैं।	
अतिरिक्त memory की जरूरत नहीं होती।	Cluster बढ़ा होने पर collisions बहुत बढ़ जाती हैं।	
Cache performance अच्छी होती है क्योंकि consecutive locations access होती हैं।	Search और Insert की performance धीरे-धीरे खराब हो जाती है।	
छोटे data set में अच्छा काम करता है।	Different hash values वाली keys भी उसी cluster में फँस जाती हैं।	
Advantages (Secondary Clustering / Quadratic Probing)	Disadvantages	
Primary Clustering नहीं होती।	Secondary Clustering होती है।	
Continuous cluster नहीं बनते।	Same initial hash वाली keys एक ही probe sequence follow करती हैं।	
Linear Probing से कम collisions होती हैं।	Table के सभी slots तक पहुँचना हमेशा संभव नहीं होता (गलत table size/load factor पर)।	
Search और Insert की performance Linear Probing से बेहतर होती है।	Implementation Linear Probing से थोड़ी complex होती है।	

अगर **Double Hashing** के **Advantages** और **Disadvantages** पूछे जाएँ, तो ये लिखो:

Advantages (Double Hashing)	Disadvantages
Primary Clustering नहीं होती।	दो Hash Functions बनाने पड़ते हैं।
Secondary Clustering भी नहीं होती।	सही Prime Number ($\text{prime} < m$) चुनना जरूरी होता है।
हर key का probing sequence अलग होता है।	Implementation सबसे complex होती है।
Open Addressing में सबसे अच्छा collision resolution माना जाता है।	Collision पर दूसरा hash भी calculate करना पड़ता है, इसलिए computation थोड़ी अधिक होती है।
Search और Insert की performance सबसे बेहतर रहती है (जब load factor कम हो)।	गलत second hash function चुनने पर performance खराब हो सकती है।



अगर **Separate Chaining** के **Advantages** और **Disadvantages** पूछे जाएँ, तो ये लिखो:

Advantages (Separate Chaining)	Disadvantages
Primary Clustering नहीं होती।	Extra memory की जरूरत होती है (Linked List/Vector के लिए)।
Secondary Clustering नहीं होती।	Dynamic memory allocation की वजह से overhead बढ़ता है।
Table पूरी भर जाने पर भी insertion संभव है (नई node chain में जोड़ देते हैं)।	Cache performance खराब होती है क्योंकि nodes अलग-अलग memory में हो सकते हैं।
Deletion आसान होती है (सिर्फ chain से node हटानी होती है)।	अगर एक bucket में बहुत सारी keys आ जाएँ, तो search $O(n)$ तक पहुँच सकती है।
Load Factor (α) 1 से बड़ा भी हो सकता है।	Worst case में सभी keys एक ही bucket में आ जाएँ तो पूरी linked list traverse करनी पड़ती है।